

Praca projektowa (inżynierska)

Uczenie maszynowe w cyberbezpieczeństwie chmury

Porównanie skuteczności metod uczenia głębokiego i lasów losowych w prognozowaniu naruszeń danych w chmurze

*Comparison of deep learning methods and random forests in forecasting
cloud data breaches*

Temat: Detekcja i prognozowanie naruszeń danych

Metody: Lasy losowe vs. sieci głębokie (MLP, CNN, LSTM)

Zbiór danych: CSE-CIC-IDS2018 / dane syntetyczne

30 maja 2026

Streszczenie

Coraz więcej danych trafia do chmury obliczeniowej, a wraz z nimi rośnie ryzyko ich naruszenia. Celem tej pracy jest sprawdzenie, jak dobrze metody uczenia maszynowego potrafią wykrywać ataki prowadzące do takich naruszeń, oraz porównanie dwóch popularnych podejść: klasycznego *lasu losowego* oraz *sieci głębokich* (MLP, CNN i LSTM). Zadanie sformułowano jako rozpoznawanie złośliwych połączeń sieciowych — model decyduje, czy dane połączenie to ruch normalny, czy atak.

W ramach projektu zbudowano kompletny, samodzielnie uruchamialny program w języku Python, który wczytuje dane, przygotowuje je, trenuje wszystkie modele i porównuje ich wyniki. Eksperyment przeprowadzono na danych *cicids2018*, opartych na zbiorze CSE-CIC-IDS2018 — ruchu sieciowym zebrany na infrastrukturze chmurowej. Modele oceniono zarówno pod kątem skuteczności (m.in. wykrywalność ataków i liczba fałszywych alarmów), jak i kosztu (czas uczenia oraz złożoność).

Wyniki pokazują, że najlepszy model głęboki (CNN 1D, $F1 = 0.913$) osiąga nieco wyższą skuteczność niż las losowy ($F1 = 0.912$), ale kosztem znacznie dłuższego uczenia. Las losowy okazuje się najlepszym kompromisem między jakością a kosztem, a bardziej skomplikowane sieci (CNN, LSTM) nie dają tutaj wyraźnej przewagi.

Słowa kluczowe: bezpieczeństwo chmury, naruszenie danych, wykrywanie ataków, uczenie maszynowe, las losowy, sieci głębokie, CSE-CIC-IDS2018.

Spis treści

Streszczenie	1
1 Wstęp	4
1.1 Wprowadzenie	4
1.2 Cel pracy	4
1.3 Zakres i sposób realizacji	4
1.4 Układ dokumentu	5
2 Bezpieczeństwo danych w chmurze obliczeniowej i sformułowanie problemu	5
2.1 Specyfika zagrożeń w środowisku chmurowym	5
2.2 Pojęcie naruszenia danych	5
2.3 Od wykrywania ataków do prognozowania naruszeń	6
2.4 Sformułowanie zadania klasyfikacji	6
2.5 Wyzwania badawcze	6
3 Podstawy teoretyczne zastosowanych metod	7
3.1 Las losowy	7
3.2 Sieci głębokie	7
3.3 Miary oceny skuteczności	8
3.4 Przegląd literatury	8
4 Metodyka badawcza	9
4.1 Ogólny schemat eksperymentu	9
4.2 Zbiór danych	9
4.3 Przetwarzanie wstępne danych	10
4.4 Konfiguracja modeli	10
4.5 Miary oceny i środowisko badawcze	11
5 Elementy realizacji zadania	11
5.1 Implementacja potoku	11
5.2 Skuteczność modeli	12
5.3 Analiza błędów	13
5.4 Wydajność i koszt obliczeniowy	14
5.5 Przebieg uczenia modeli głębokich	15
5.6 Interpretowalność modeli i ważność cech	16
5.7 Podsumowanie wyników	16
6 Wnioski końcowe	17
6.1 Podsumowanie	17
6.2 Ograniczenia	17

6.3	Kierunki dalszych prac	18
6.4	Podsumowanie końcowe	18

1 Wstęp

1.1 Wprowadzenie

Trudno dziś wyobrazić sobie firmę, która nie korzysta z chmury obliczeniowej. Poczta, dokumenty, bazy danych, całe aplikacje — wszystko to coraz częściej działa na serwerach wynajmowanych od dostawców takich jak Amazon, Microsoft czy Google. Razem z usługami do chmury trafiają jednak dane, w tym te najbardziej wrażliwe. To czyni chmurę łakomym kąskiem dla cyberprzestępców, a *naruszenie danych* — czyli ich wyciek lub kradzież — jednym z najbardziej kosztownych incydentów, jakie mogą spotkać organizację.

Klasyczne zabezpieczenia, oparte na gotowych „sygnaturach” znanych ataków, coraz słabiej radzą sobie z nowymi zagrożeniami i z ogromną skalą ruchu w chmurze. Dlatego dużą nadzieję pokłada się w *uczeniu maszynowym* — czyli w modelach, które samodzielnie uczą się rozpoznawać złośliwą aktywność na podstawie wcześniejszych przykładów. Wśród wielu dostępnych metod szczególnie wyróżniają się dwie: *lasy losowe* (sprawdzona, „klasyczna” technika) oraz *uczenie głębokie* (nowsze, oparte na sieciach neuronowych). Obie potrafią osiągać świetne wyniki, ale różnią się złożonością, kosztem i tym, jak łatwo zrozumieć ich decyzje.

1.2 Cel pracy

Celem pracy jest **porównanie skuteczności metod uczenia głębokiego i lasów losowych w wykrywaniu (prognozowaniu) naruszeń danych w chmurze**. Przez „prognozowanie naruszenia” rozumiemy rozpoznawanie pojedynczych połączeń sieciowych jako normalnych lub złośliwych — wykrycie podejrzanego połączenia jest bowiem wczesnym ostrzeżeniem przed pełnym wyciekiem danych.

Chcemy w szczególności odpowiedzieć na trzy pytania:

1. Które podejście — las losowy czy sieci głębokie — skuteczniej wykrywa ataki?
2. Ile kosztuje (pod względem czasu uczenia i działania) każde z nich i czy wyższa skuteczność jest warta tego kosztu?
3. Czy bardziej rozbudowane sieci (CNN, LSTM) są lepsze od prostej sieci MLP na danych opisujących ruch sieciowy?

1.3 Zakres i sposób realizacji

Projekt ma charakter zarówno teoretyczny, jak i praktyczny. Część praktyczna to napisany od podstaw program w Pythonie, który realizuje cały eksperyment — od przygotowania danych, przez uczenie pięciu modeli, po obliczenie wyników i wygenerowanie wykresów. Wszystkie liczby i rysunki w sprawozdaniu pochodzą bezpośrednio z tego programu, dzięki czemu treść jest zgodna z faktycznymi wynikami.

1.4 Układ dokumentu

Dalsza część pracy ma typowy układ: w rozdziale 2 przybliżamy problem bezpieczeństwa danych w chmurze, w rozdziale 3 opisujemy porównywane metody, rozdział 4 przedstawia sposób przeprowadzenia eksperymentu, rozdział 5 prezentuje i omawia wyniki, a rozdział 6 zawiera wnioski oraz pomysły na dalsze prace.

2 Bezpieczeństwo danych w chmurze obliczeniowej i sformułowanie problemu

2.1 Specyfika zagrożeń w środowisku chmurowym

Chmura obliczeniowa zmieniła sposób, w jaki firmy przechowują i przetwarzają dane. Zamiast utrzymywać własne serwerownie, organizacje wynajmują zasoby od dostawców takich jak Amazon Web Services, Microsoft Azure czy Google Cloud. Wygoda i oszczędność mają jednak swoją cenę — dane wielu klientów żyją na tej samej, współdzielonej infrastrukturze, dostępnej przez sieć z dowolnego miejsca na świecie. To sprawia, że chmura jest atrakcyjnym celem dla atakujących.

Warto pamiętać o tzw. **modelu współdzielonej odpowiedzialności**: dostawca chmury dba o bezpieczeństwo samej infrastruktury, ale za poprawną konfigurację, kontrolę dostępu i ochronę własnych danych odpowiada już klient. W praktyce ogromna część głośnych wycieków danych z chmury wynikała nie z włamania do serwerów dostawcy, lecz z błędów po stronie użytkownika — źle ustawionych uprawnień, otwartych „kubeków” z danymi czy przejętych kont. Dodatkowo skala i zmienność ruchu w chmurze są tak duże, że ręczne pilnowanie bezpieczeństwa przestaje być możliwe.

2.2 Pojęcie naruszenia danych

Naruszenie danych (ang. *data breach*) to sytuacja, w której dane trafiają w niepowołane ręce — zostają wykradzione, ujawnione lub zmienione bez uprawnienia. Konsekwencje bywają dotkliwe: kary finansowe (np. na gruncie RODO), utrata zaufania klientów i realne straty. Co gorsza, naruszenia często pozostają niewykryte przez wiele tygodni, co daje atakującym czas na wyrządzenie dużych szkód.

W środowisku chmurowym najczęstsze drogi prowadzące do naruszenia to m.in.:

- **przejęcie kont** — np. zgadywanie haseł metodą brute force do usług logowania;
- **eksfiltracja danych** — niepostrzeżone wyprowadzanie danych na zewnątrz, często ukryte w pozornie zwykłym ruchu;
- **ataki DoS/DDoS** — zalewanie usług ruchem, czasem jako zasłona dymna dla innych działań;

- **infiltracja i ruch boczny** — po zdobyciu przyczółka atakujący przemieszcza się w infrastrukturze ofiary;
- **botnety oraz ataki na aplikacje webowe** (np. SQL injection), które również prowadzą do wycieku danych.

Wszystkie te działania mają jedną wspólną cechę: zostawiają ślad w *ruchu sieciowym*. Analizując ten ruch, można wychwycić podejrzaną aktywność, zanim przerodzi się ona w pełne naruszenie.

2.3 Od wykrywania ataków do prognozowania naruszeń

Naruszenie danych rzadko jest pojedynczym zdarzeniem — to zwykle finał całego łańcucha działań: rozpoznanie, włamanie, eskalacja uprawnień, wreszcie kradzież danych. Jeśli uda się wykryć wczesne ogniwa tego łańcucha (np. nietypowe połączenie świadczące o skanowaniu lub łamaniu haseł), można zareagować, *zanim* dojdzie do wycieku. W tym sensie wykrycie ataku pełni rolę *prognostyczną* — jest wczesnym sygnałem ostrzegawczym. Dlatego w tej pracy „prognozowanie naruszenia” utożsamiamy z rozpoznawaniem złośliwych połączeń sieciowych.

2.4 Sformułowanie zadania klasyfikacji

Zadanie sprowadza się do prostego pytania zadawanego dla każdego połączenia sieciowego: *czy to zwykły, normalny ruch, czy raczej atak?* Każde połączenie opisujemy zestawem liczb — na przykład jak długo trwało, ile pakietów i bajtów przesłano, jak często, jakie flagi sieciowe się pojawiły. Na podstawie tych cech model ma przykleić jedną z dwóch etykiet: *ruch normalny* albo *atak*. Model uczy się tej umiejętności na historycznych przykładach, dla których wiadomo, co było czym. Jest to więc klasyczne zadanie **klasyfikacji** — a ponieważ dane zawierają też informację o rodzaju ataku, możliwa jest dodatkowo analiza poszczególnych typów zagrożeń.

2.5 Wyzwania badawcze

Wykrywanie naruszeń nie jest proste z kilku praktycznych powodów:

- **Ataki są rzadkie.** W normalnym ruchu tonie garstka złośliwych połączeń. Model łatwo wpada w pułapkę uznawania „wszystkiego za normalne” i przeocza zagrożenia.
- **Nie każdy błąd kosztuje tyle samo.** Przeoczenie ataku jest zwykle znacznie groźniejsze niż fałszywy alarm — dlatego zależy nam przede wszystkim na wykrywalności (czułości).
- **Złośliwość kryje się w kombinacjach.** Pojedyncza cecha rzadko zdradza atak; dopiero ich zestawienie tworzy podejrzaną obraz, co wyklucza najprostsze, „liniowe” metody.

- **Liczy się szybkość i koszt.** W chmurze model trzeba często dotrenowywać i stosować na bieżąco do ogromnych ilości ruchu, więc nie może być zbyt kosztowny obliczeniowo.

Te właśnie wyzwania kształtują dobór metod i sposób ich oceny, co opisujemy w kolejnych rozdziałach.

3 Podstawy teoretyczne zastosowanych metod

W tym rozdziale opisano — w sposób możliwie przystępny, bez zagłębiania się w formalizm matematyczny — dwie rodziny metod, które porównujemy: las losowy oraz sieci głębokie. Celem jest zrozumienie, *jak* działają i *czym* się różnią, bo to te różnice decydują o ich przydatności w ochronie danych w chmurze.

3.1 Las losowy

Punktem wyjścia jest *drzewo decyzyjne*. Można je sobie wyobrazić jako listę pytań typu „czy liczba pakietów w połączeniu przekracza pewien próg?”. Odpowiadając na kolejne pytania, schodzimy w dół drzewa aż do liścia, który mówi: „to ruch normalny” albo „to atak”. Pojedyncze drzewo jest jednak kapryśne — drobna zmiana danych potrafi je całkowicie przebudować, przez co łatwo się „przeucza” i słabo radzi sobie z nowymi przypadkami.

Las losowy [1] rozwiązuje ten problem prostą, ale skuteczną sztuczką: zamiast jednego drzewa buduje ich setki, a każde uczy na nieco innym, losowym wycinku danych i cech. Decyzja końcowa zapada przez *głosowanie* — jeśli większość drzew uznaje połączenie za atak, takim jest klasyfikowane. Dzięki temu, że drzewa popełniają różne błędy, ich uśrednienie daje wynik znacznie stabilniejszy niż każde drzewo z osobna.

Las losowy ma kilka cech, które czynią go popularnym w cyberbezpieczeństwie: nie wymaga skalowania danych, dobrze radzi sobie ze złożonymi zależnościami między cechami, szybko się uczy i — co istotne dla analityka — potrafi wskazać, **które cechy** były najważniejsze przy podejmowaniu decyzji.

3.2 Sieci głębokie

Sieć neuronowa to model luźno inspirowany działaniem mózgu. Składa się z warstw „neuronów”, z których każdy łączy informacje z poprzedniej warstwy i przekazuje wynik dalej. Określenie *głęboka* oznacza po prostu, że warstw jest wiele — dzięki temu sieć stopniowo buduje coraz bardziej złożone wzorce: od prostych zależności po abstrakcyjne „sygnatury” złośliwej aktywności. Sieć uczy się, wielokrotnie oglądając przykłady i samodzielnie korygując swoje „wewnętrzne pokręta” tak, aby coraz rzadziej się mylić.

W pracy porównujemy trzy architektury sieci:

MLP (perceptron wielowarstwowy). Najprostsza i najbardziej klasyczna sieć w pełni połączona. Traktuje dane jak zwykłą tabelę liczb i jest naturalnym wyborem dla danych opisujących połączenia sieciowe.

CNN 1D (sieć konwolucyjna). Sieć, która „przesuwa okienko” po danych i wyszukuje powtarzające się, lokalne wzorce [4]. Sprawdza się świetnie w analizie obrazów czy sygnałów; tutaj sprawdzamy, czy podobne podejście pomaga w danych o ruchu sieciowym.

LSTM (sieć rekurencyjna). Sieć z „pamięcią”, zaprojektowana do danych o charakterze sekwencji [3]. Potrafi uwzględniać kolejność i zależności między kolejnymi elementami, dlatego włączamy ją do porównania, by sprawdzić, czy modelowanie „sekwencyjności” cech przynosi jakkolwiek korzyść.

3.3 Miary oceny skuteczności

Sama „dokładność” (odsetek poprawnych decyzji) bywa myląca, gdy ataki są rzadkie — model, który wszystko uzna za ruch normalny, może mieć wysoką dokładność, a jednocześnie nie wykryć żadnego włamania. Dlatego patrzymy na kilka uzupełniających się miar, opisanych tu zwykłym językiem:

- **Precyzja** — spośród połączeń oznaczonych jako atak, jaki odsetek faktycznie był atakiem. Niska precyzja oznacza wiele *fałszywych alarmów*, które męczą zespół bezpieczeństwa.
- **Czułość** (ang. *recall*) — spośród wszystkich prawdziwych ataków, ile udało się wykryć. Niska czułość oznacza *przeoczone włamania* — najgroźniejszy rodzaj błędu.
- **F1** — jedna liczba godząca precyzję i czułość; wygodna do porównań.
- **ROC-AUC i PR-AUC** — miary opisujące jakość modelu niezależnie od ustawionego „progu czułości”; im bliżej 1,0, tym lepiej.

W kontekście ochrony danych w chmurze szczególnie zależy nam na *wysokiej czułości* (żeby nie przeoczyć naruszenia) przy rozsądnej precyzji (żeby nie zasypać analityków fałszywymi alarmami).

3.4 Przegląd literatury

Wykrywanie włamań metodami uczenia maszynowego to dobrze zbadany temat. Najczęściej wykorzystuje się publiczne zbiory danych, takie jak NSL-KDD [7], UNSW-NB15 [5] czy CIC-IDS2017/CSE-CIC-IDS2018 przygotowane przez Canadian Institute for Cybersecurity [6]. Co ciekawe, w wielu badaniach metody oparte na drzewach (jak las losowy) okazują się równie dobre lub lepsze od sieci głębokich, gdy dane mają postać *tabeli* — a tak właśnie wyglądają cechy połączeń sieciowych [2]. Sieci głębokie błyszczą przede wszystkim tam,

gdzie dane są surowe i mają wyraźną strukturę (obrazy, dźwięk, tekst). Niniejsza praca sprawdza, czy ta obserwacja potwierdza się również w naszym, „chmurowym” zadaniu.

4 Metodyka badawcza

4.1 Ogólny schemat eksperymentu

Aby porównanie było uczciwe, wszystkie modele uczono i testowano *na tych samych danych* i oceniano *tymi samymi miarami*. Cały eksperyment przebiega według prostego scenariusza:

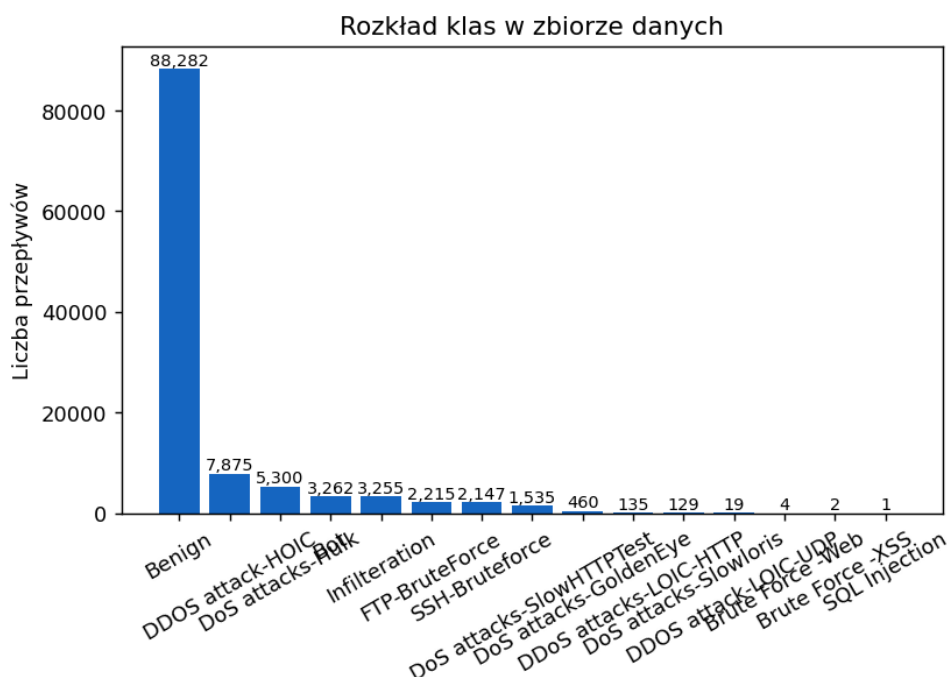
1. wczytanie danych (rzeczywisty zbiór CSE-CIC-IDS2018 lub — gdy nie jest dostępny — dane syntetyczne),
2. przygotowanie danych (oczyszczenie, ujednolicenie, podział na zbiory),
3. wytrenowanie pięciu modeli: regresji logistycznej, lasu losowego oraz sieci MLP, CNN i LSTM,
4. ocena każdego modelu (skuteczność oraz czas uczenia i działania),
5. zebranie wyników w postaci tabel i wykresów.

4.2 Zbiór danych

Głównym zbiorem danych jest **CSE-CIC-IDS2018** — duży, publicznie dostępny zbiór ruchu sieciowego zebranego na chmurowej infrastrukturze AWS przez Canadian Institute for Cybersecurity [6]. Zawiera on zarówno ruch normalny, jak i różne typy ataków (m.in. łamanie haseł, DoS/DDoS, ataki na aplikacje webowe, infiltrację oraz botnety). Każde połączenie opisane jest kilkudziesięcioma cechami liczbowymi — na przykład czasem trwania, liczbą i rozmiarem przesłanych pakietów, tempem transmisji czy licznikami flag sieciowych.

Pełny zbiór waży kilka gigabajtów, dlatego program wczytuje go w częściach i losowo próbkuję, biorąc porcję danych z każdego dnia (każdy dzień zawiera inne typy ataków). Dla zachowania pełnej powtarzalności — gdyby ktoś chciał uruchomić projekt bez pobierania wielkich plików — przygotowano też *generator danych syntetycznych* o zbliżonej charakterystyce. Program automatycznie korzysta z danych rzeczywistych, jeśli są dostępne, a w przeciwnym razie z danych syntetycznych.

W opisywanym eksperymencie wykorzystano dane **cicids2018**: łącznie 114 621 połączeń opisanych 68 cechami. Rozkład klas (widoczna przewaga ruchu normalnego, typowa dla takich zadań) przedstawia rys. 1.



Rysunek 1: Rozkład klas w danych. Ruch normalny wyraźnie dominuje nad atakami — to częste wyzwanie w wykrywaniu zagrożeń.

4.3 Przetwarzanie wstępne danych

Surowe dane wymagają porządków. Etap przygotowania obejmuje:

- usunięcie kolumn nieprzydatnych do klasyfikacji (adresy IP, porty, znaczniki czasu) oraz kolumn o stałej wartości;
- poprawienie brakujących i błędnych wartości (np. wartości nieskończonych powstających przy dzieleniu);
- zamianę etykiet na prostą postać: ruch normalny lub atak (zachowano też informację o konkretnym typie ataku do celów poglądowych);
- ujednolicenie skali cech, tak aby duże liczby nie „zagłuszały” mniejszych — co jest ważne zwłaszcza dla sieci neuronowych;
- podział danych na trzy części: uczącą, walidacyjną (do strojenia) i testową (do końcowej, uczciwej oceny).

Podział wykonano z ustalonym ziarnem losowości, dzięki czemu wyniki są powtarzalne. Liczności zbiorów to: 73 356 (uczący), 18 340 (walidacyjny) i 22 925 (testowy) połączeń.

4.4 Konfiguracja modeli

Porównano pięć modeli o rosnącej złożoności:

- **Regresja logistyczna** — prosty model „liniowy”, traktowany jako punkt odniesienia: pokazuje, ile zyskujemy, sięgając po metody bardziej zaawansowane.
- **Las losowy** — zespół 300 drzew decyzyjnych.
- **MLP** — klasyczna sieć z dwiema warstwami ukrytymi.
- **CNN 1D** — sieć konwolucyjna z dwiema warstwami filtrów.
- **LSTM** — dwukierunkowa sieć rekurencyjna.

Aby uczciwie potraktować rzadkość ataków, w każdym modelu zastosowano mechanizmy równoważące klasy (model jest „karany” mocniej za przeoczenie ataku). Sieci głębokie uczono z wykorzystaniem *wczesnego zatrzymania* — nauka kończy się, gdy model przestaje się poprawiać na danych walidacyjnych, co zapobiega przeuczeniu.

4.5 Miary oceny i środowisko badawcze

Skuteczność oceniono miarami opisanymi w rozdziale 3 (dokładność, precyzja, czułość, F1, ROC-AUC i PR-AUC). Dodatkowo zmierzono *koszt*: czas potrzebny na wytrenowanie modelu, czas potrzebny na sklasyfikowanie danych testowych oraz złożoność modelu (liczba parametrów lub węzłów drzew). Całość zaimplementowano w języku **Python** przy użyciu bibliotek **scikit-learn** (modele klasyczne) oraz **PyTorch** (sieci głębokie); wykresy przygotowano w **matplotlib**. Obliczenia prowadzono na zwykłym procesorze (CPU), co dobrze oddaje warunki ograniczonych zasobów i uwypukla różnice w koszcie między metodami.

5 Elementy realizacji zadania

5.1 Implementacja potoku

Część praktyczną zrealizowano jako modułowy potok w języku Python (katalog `src/`). Najważniejsze komponenty to:

- `data/` — generator danych syntetycznych, wczytywanie realnego zbioru CIC-IDS2018 oraz przetwarzanie wstępne (czyszczenie, kodowanie, standaryzacja, stratyfikowany podział);
- `models/classical.py` — las losowy i regresja logistyczna (**scikit-learn**);
- `models/deep.py` — architektury MLP, CNN 1D i LSTM (PyTorch) z wspólną pętlą treningową, wczesnym zatrzymaniem i ważeniem klas;
- `evaluate.py`, `plots.py`, `tables.py` — obliczanie metryk, generowanie rysunków oraz tabel i makr $\text{\LaTeX}$;

- `run_all.py` — punkt wejścia uruchamiający cały eksperyment.

Wszystkie wyniki liczbowe, tabele i rysunki w tym rozdziale zostały wygenerowane automatycznie przez ten kod, dzięki czemu treść sprawozdania jest ściśle spójna z implementacją.

5.2 Skuteczność modeli

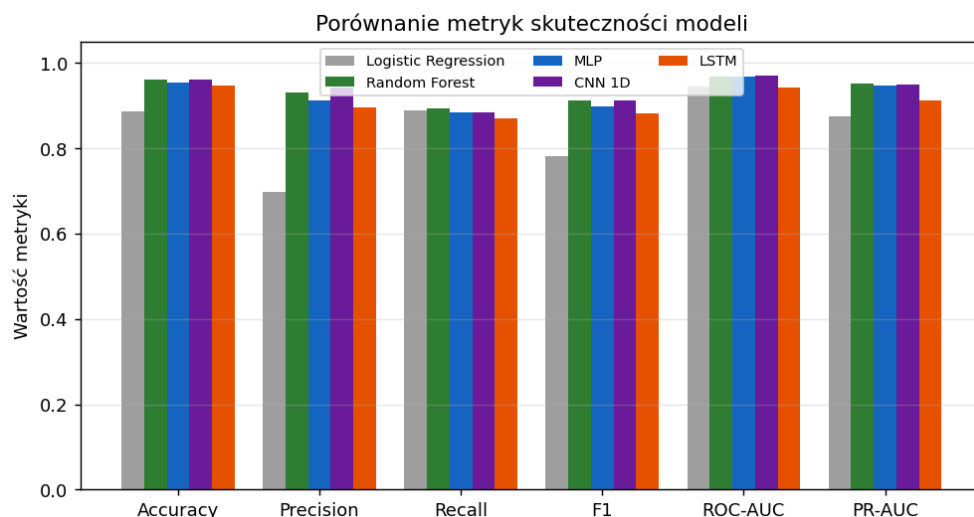
Tabela 1 zestawia metryki skuteczności wszystkich modeli na zbiorze testowym (wartości najlepsze w każdej kolumnie wyróżniono pogrubieniem).

Tabela 1: Porównanie skuteczności modeli na zbiorze testowym.

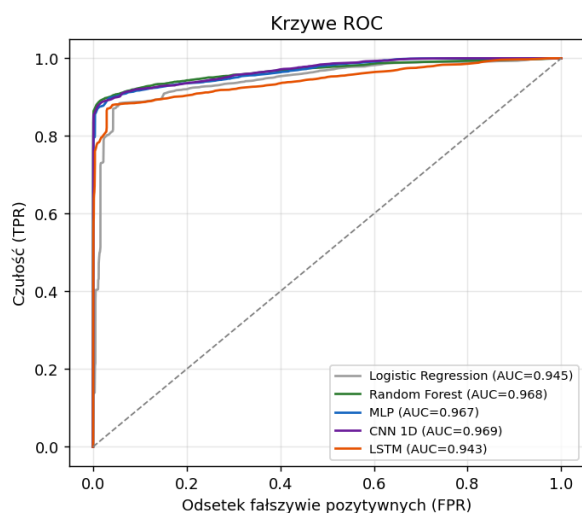
Model	Accuracy	Precision	Recall	F1	ROC-AUC	PR-AUC
Logistic Regression	0.886	0.699	0.890	0.783	0.945	0.875
Random Forest	0.961	0.932	0.893	0.912	0.968	0.953
MLP	0.954	0.912	0.884	0.898	0.967	0.946
CNN 1D	0.961	0.942	0.885	0.913	0.969	0.949
LSTM	0.947	0.895	0.870	0.882	0.943	0.913

Z zestawienia wynika kilka obserwacji. Po pierwsze, wszystkie metody bardziej zaawansowane wyraźnie przewyższają prostą regresję logistyczną — potwierdza to, że atak zdradza dopiero *kombinacja* cech, a nie pojedyncze wartości. Po drugie, na samym szczycie panuje praktycznie remis: najlepszy model głęboki **CNN 1D** ($F_1 = 0.913$) i **las losowy** ($F_1 = 0.912$, ROC-AUC = 0.968) osiągają niemal identyczne wyniki — różnica jest na granicy szumu. Po trzecie, oba modele mają nieco inny „charakter”: las losowy uzyskuje najwyższą *czułość* (najwięcej wykrytych ataków) oraz najwyższy PR-AUC, podczas gdy CNN ma najwyższą *precyzję* (najmniej fałszywych alarmów). Warto też zauważyć, że najbardziej rozbudowany model — rekurencyjna sieć LSTM — wypada najsłabiej spośród metod nieliniowych, mimo największej złożoności.

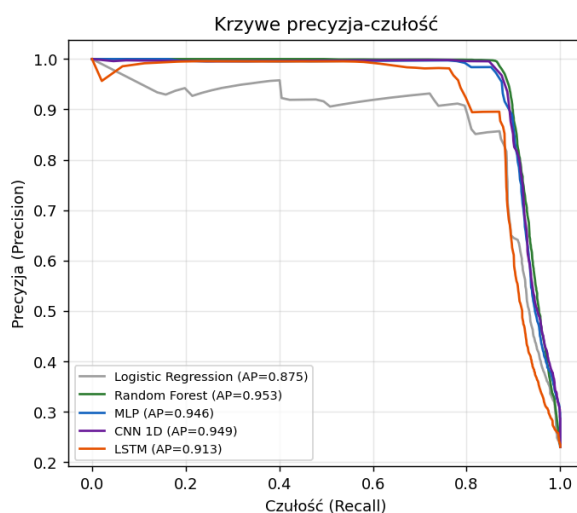
Graficzne porównanie metryk przedstawia rys. 2, a krzywe ROC i precyzja–czułość — rys. 3 i 4. Krzywe ROC dla wszystkich modeli nieliniowych przebiegają blisko lewego górnego rogu, co świadczy o dobrej separowalności klas.



Rysunek 2: Porównanie metryk skuteczności modeli na zbiorze testowym.



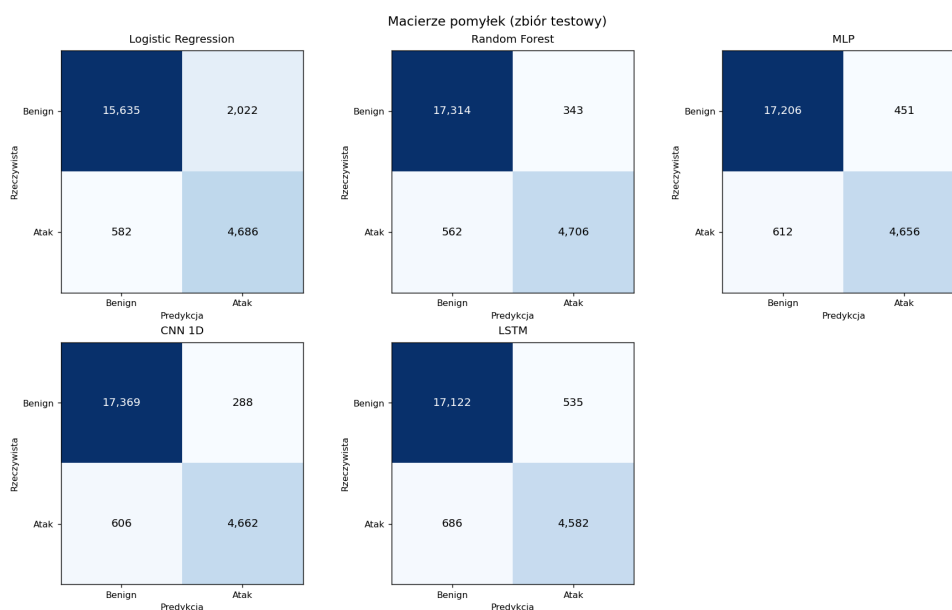
Rysunek 3: Krzywe ROC.



Rysunek 4: Krzywe precyzja-czułość.

5.3 Analiza błędów

Macierze pomyłek (rys. 5) pozwalają zajrzeć „pod maskę” i zobaczyć, jakie błędy popełniają modele. Las losowy wychwytuje najwięcej ataków (najwyższa czułość), czyli rzadziej przepuszcza zagrożenie — co w kontekście ochrony danych jest cenne, bo przeoczone naruszenie kosztuje najwięcej. Sieć CNN z kolei nieco rzadziej podnosi fałszywy alarm (najwyższa precyzja), czyli mniej męczy zespół bezpieczeństwa niepotrzebnymi powiadomieniami. W praktyce „punkt równowagi” między tymi błędami można regulować progiem decyzyjnym, dopasowując go do polityki bezpieczeństwa danej organizacji.



Rysunek 5: Macierze pomyłek na zbiorze testowym dla wszystkich modeli.

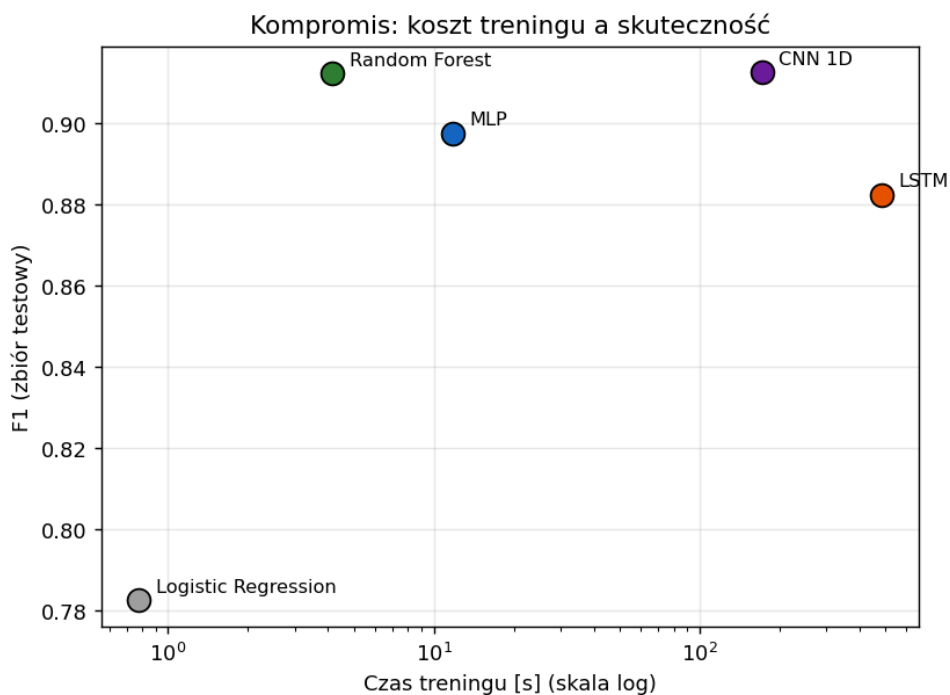
5.4 Wydajność i koszt obliczeniowy

Skuteczność to tylko jeden wymiar oceny; w środowisku chmurowym, gdzie modele są często dotrenowywane i muszą klasyfikować ruch na bieżąco, kluczowy jest także koszt. Tabela 2 zestawia czasy treningu i inferencji oraz złożoność modeli.

Tabela 2: Porównanie wydajności i złożoności modeli.

Model	Czas treningu [s]	Czas inferencji [s]	Złożoność modelu
Logistic Regression	0.78	0.0006	69 wsp.
Random Forest	4.16	0.0801	300 drzew / 1 550 456 węzłów
MLP	11.82	0.0281	17 537 param.
CNN 1D	171.74	0.7103	39 553 param.
LSTM	487.45	3.9718	134 401 param.

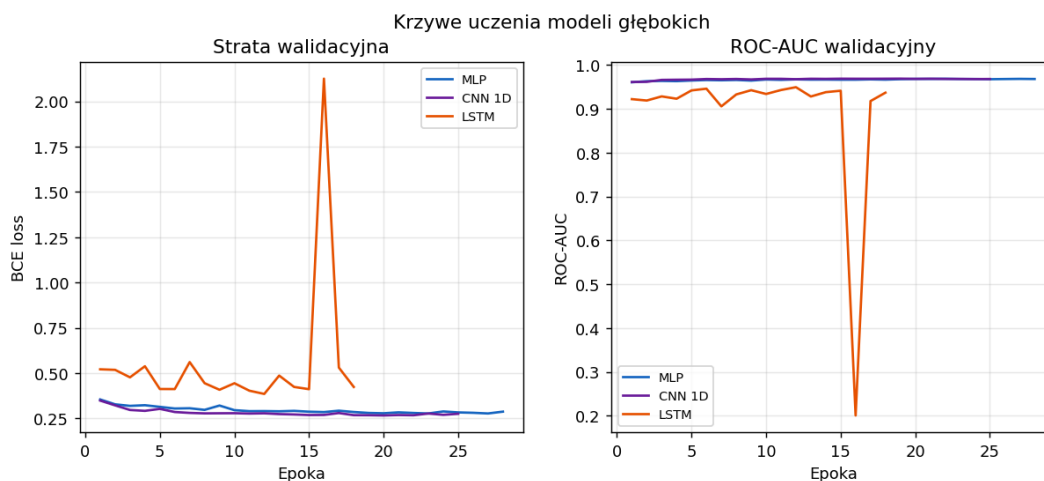
I tu pojawia się sedno sprawy. Las losowy uczy się w zaledwie 4.16 s, podczas gdy najlepszy model głęboki (CNN 1D) potrzebuje aż 171.74 s — czyli kilkadziesiąt razy dłużej — by osiągnąć praktycznie ten sam wynik. Najbardziej rozbudowana sieć LSTM trenuje się ponad osiem minut, a mimo to wypada słabiej. Również w fazie działania (klasyfikacji nowego ruchu) las losowy i proste modele są wyraźnie szybsze od sieci CNN i LSTM. Ten kompromis między kosztem a skutecznością dobrze obrazuje rys. 6: najbardziej opłacalne modele leżą w lewej górnej części wykresu — wysoka skuteczność przy krótkim czasie uczenia.



Rysunek 6: Kompromis między czasem treningu (skala logarytmiczna) a skutecznością (F_1). Modele najbardziej efektywne znajdują się w lewym górnym obszarze.

5.5 Przebieg uczenia modeli głębokich

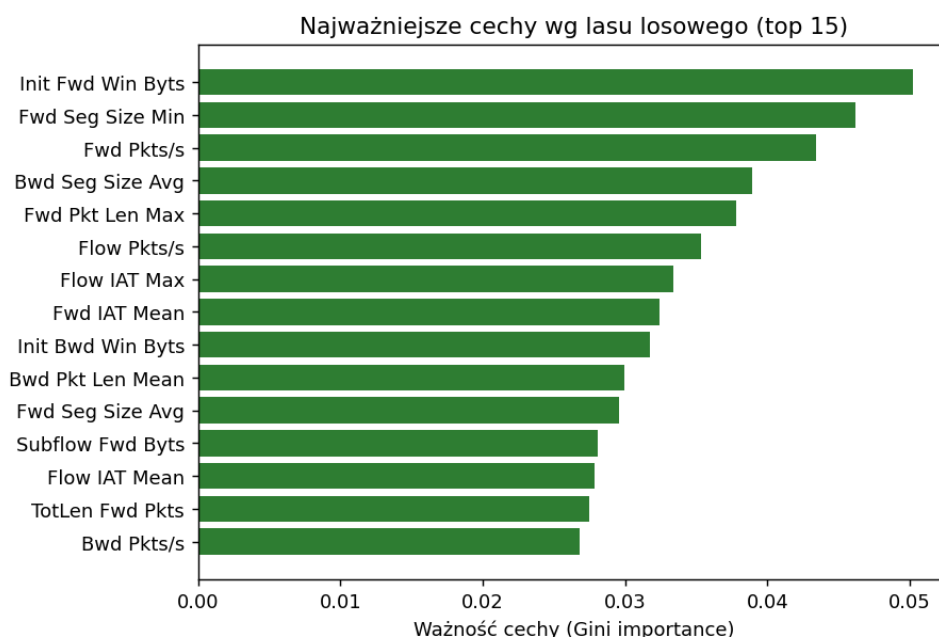
Rys. 7 przedstawia krzywe uczenia modeli głębokich: stratę walidacyjną oraz ROC-AUC na zbiorze walidacyjnym w kolejnych epokach. Widoczna jest stabilna zbieżność i działanie mechanizmu wczesnego zatrzymania, który zapobiega przeuczeniu.



Rysunek 7: Krzywe uczenia modeli głębokich (strata i ROC-AUC walidacyjny).

5.6 Interpretowalność modeli i ważność cech

Dodatkową zaletą lasu losowego jest interpretowalność. Rys. 8 przedstawia najważniejsze cechy według miary ważności Giniego. Pozwala to zidentyfikować, które statystyki przepływu najsilniej różnicują ruch złośliwy i normalny — informacja cenna dla analityków bezpieczeństwa, a trudniej dostępna w modelach głębokich, działających jak „czarna skrzynka”.



Rysunek 8: Najważniejsze cechy według lasu losowego (ważność Giniego).

5.7 Podsumowanie wyników

Zebrane wyniki pozwalają wstępnie odpowiedzieć na postawione pytania:

- **Skuteczność:** las losowy i najlepszy model głęboki (CNN 1D) są praktycznie nierozróżnialne pod względem F_1 i ROC-AUC; las losowy dodatkowo wykrywa najwięcej ataków (najwyższa czułość).
- **Koszt:** las losowy jest zdecydowanie tańszy — uczy się kilkadziesiąt razy szybciej niż sieci głębokie, oferując najlepszą relację skuteczność–koszt.
- **Złożoność:** rozbudowane sieci sekwencyjne (zwłaszcza LSTM) nie przyniosły przewagi — większa złożoność nie przełożyła się na lepsze wyniki, co potwierdza obserwacje z literatury dotyczące danych tabelarycznych.

6 Wnioski końcowe

6.1 Podsumowanie

W pracy zrealizowano empiryczne porównanie metod uczenia głębokiego i lasów losowych w zadaniu prognozowania naruszeń danych w chmurze, sformułowanym jako binarna klasyfikacja przepływów sieciowych. Zbudowano kompletny, odtwarzalny potok obejmujący przygotowanie danych, trening pięciu modeli (regresja logistyczna, las losowy oraz sieci MLP, CNN 1D i LSTM) oraz ich ocenę pod kątem skuteczności i kosztu obliczeniowego.

Najważniejsze wnioski są następujące:

1. **Obie rodziny metod skutecznie wykrywają naruszenia.** Zarówno las losowy, jak i sieci głębokie osiągają wysokie wyniki, wyraźnie przewyższając prosty model liniowy. To pokazuje, że złośliwa aktywność kryje się w złożonych zależnościach między cechami, które obie metody dobrze wychytują.
2. **Skuteczność jest niemal taka sama, ale koszt — już nie.** Najlepszy model głęboki (CNN 1D, $F_1 = 0.913$) i las losowy ($F_1 = 0.912$) osiągają praktycznie identyczną jakość, przy czym las losowy dodatkowo wykrywa najwięcej ataków (najwyższa czułość), uczy się kilkadziesiąt razy szybciej i potrafi wskazać, które cechy zaważyły na decyzji. To czyni go wyjątkowo atrakcyjnym wyborem.
3. **Bardziej skomplikowane nie znaczy lepsze.** Rozbudowane sieci sekwencyjne (zwłaszcza LSTM) nie przewyższyły ani lasu losowego, ani prostszej sieci, generując jedynie wyższy koszt. Potwierdza to znaną z literatury obserwację, że dla danych w postaci tabeli metody oparte na drzewach pozostają bardzo trudne do pobicia.
4. **Wybór metody zależy od kontekstu.** Jeśli liczą się szybkie dotrenowywanie, niski koszt, dobra wykrywalność i zrozumiałość decyzji — las losowy jest naturalnym wyborem. Sieci głębokie warto rozważyć tam, gdzie zależy nam na maksymalnym ograniczeniu fałszywych alarmów (wysoka precyzja CNN) i dysponujemy większymi zasobami obliczeniowymi.

6.2 Ograniczenia

Wyniki należy interpretować w świetle ograniczeń pracy:

- Eksperyment przeprowadzono na losowym podzbiorze danych `cicids2018` (ze względu na rozmiar pełnego zbioru liczącego kilka gigabajtów). Większa próbka mogłaby nieco zmienić konkretne liczby, choć raczej nie ogólne wnioski; program jest gotowy do uruchomienia na pełnym zbiorze bez zmian w kodzie.
- Nie przeprowadzono wyczerpującego strojenia hiperparametrów — użyto rozsądnych wartości domyślnych, jednakowych dla wszystkich modeli.

- Obliczenia prowadzono na CPU; na akceleratorach GPU względne czasy treningu modeli głębokich uległyby skróceniu, choć relacja kosztów pozostałaby jakościowo podobna.
- Zadanie ograniczono do klasyfikacji binarnej; analiza wieloklasowa (rozróżnianie typów ataków) stanowi naturalne rozszerzenie.

6.3 Kierunki dalszych prac

Możliwe rozszerzenia obejmują: (1) walidację na pełnym zbiorze CSE-CIC-IDS2018 i innych zbiorach (UNSW-NB15, NSL-KDD); (2) systematyczne strojenie hiperparametrów oraz włączenie metod gradient boosting (XGBoost, LightGBM); (3) klasyfikację wieloklasową i wykrywanie nieznanych ataków metodami półnadzorowanymi; (4) analizę odporności modeli na ataki adversarialne oraz dryf danych; (5) integrację najlepszego modelu w architekturze strumieniowej (np. przetwarzanie przepływów w czasie rzeczywistym) wraz z mechanizmami wyjaśnialności (SHAP) dla wsparcia analityków SOC.

6.4 Podsumowanie końcowe

Przeprowadzone badanie pokazuje, że w prognozowaniu naruszeń danych w chmurze nie ma jednej, bezdyskusyjnie najlepszej metody — wybór jest kompromisem między skutecznością a kosztem, szybkością i zrozumiałością decyzji. Dla danych opisujących ruch sieciowy las losowy okazuje się wyjątkowo silnym i tanim rozwiązaniem, któremu sieci głębokie co najwyżej dorównują, lecz go nie prześcigają — i to dopiero za cenę znacznie większego nakładu obliczeniowego. Dla studenta czy inżyniera bezpieczeństwa płynie z tego praktyczny wniosek: zanim sięgnie się po skomplikowane sieci neuronowe, warto najpierw wypróbować dobrze dobraną metodę klasyczną.

Literatura

- [1] Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.
- [2] Léo Grinsztajn, Edouard Oyallon, and Gaël Varoquaux. Why do tree-based models still outperform deep learning on typical tabular data? *Advances in Neural Information Processing Systems (NeurIPS)*, 35, 2022.
- [3] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.
- [4] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.
- [5] Nour Moustafa and Jill Slay. Unsw-nb15: A comprehensive data set for network intrusion detection systems. In *Military Communications and Information Systems Conference (MilCIS)*, pages 1–6, 2015.
- [6] Iman Sharafaldin, Arash Habibi Lashkari, and Ali A. Ghorbani. Toward generating a new intrusion detection dataset and intrusion traffic characterization. In *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*, pages 108–116, 2018.
- [7] Mahbod Tavallaei, Ebrahim Bagheri, Wei Lu, and Ali A. Ghorbani. A detailed analysis of the kdd cup 99 data set. In *IEEE Symposium on Computational Intelligence for Security and Defense Applications (CISDA)*, pages 1–6, 2009.